

PERFECT LABELS FOR FREE? EXACT ANSWERS TEACH SHORTCUTS; EXACT TRACES TEACH STEPS SEEN OFTEN ENOUGH

Anonymous authors

Paper under double-blind review

ABSTRACT

Exact verifiers (primality tests, SAT solvers, unit tests) label data with no noise at no marginal cost, and they supply the rewards of verifiable-reward post-training. We use them as a measurement instrument: with label noise removed, the effect of the supervision *format* can be isolated. We fine-tune Qwen2.5-1.5B-Instruct with LoRA on exactly labeled examples and compare supervision by the answer alone with supervision by a deterministic procedure trace followed by the answer. Answer-only training recovers about what simple surface probes recover: on 4-digit primality it reaches 82.3% and coincides with the rule “the last digit is 1, 3, 7 or 9” on 99.6–100.0% of test items; on divisibility by 7 or 13 it stays at chance at every training size. Trace supervision teaches the procedure when each step is supervised often enough. Divisibility by 7 rises from 50.0% at 60 examples to 99.6% at 270, and divisibility by 13 from 54.6% at 180 to 94.6% at 540. A pre-registered hypothesis that per-step accuracy follows m , the number of supervised transitions per (remainder, digit) table entry, which is known before training, held in 3 of the 4 predicted cells scored so far. It failed where the step is easier (divisibility by 11), and a dose test failed where the step is harder (13 trails 7 at equal m). A preregistered prediction that trace accuracy compounds a fixed per-step accuracy over length also failed. Controls show that the trace must be the input’s own computation, must precede the answer, and is not replaced by prompting the base model. The lesson concerns supervision design for verifiable-reward pipelines, not arithmetic.

1 INTRODUCTION

Verifiable-reward post-training rests on programs that decide correctness exactly: answer checkers for mathematics, unit tests for code, solvers for logic (Lambert et al., 2024; DeepSeek-AI et al., 2025). Such verifiers are often described as a source of perfect labels for free. They are perfect in one sense: a verified label is never wrong. Whether a model trained on those labels learns the computation the verifier performs, or a cheaper function that agrees with it on the training distribution, is a separate question, and in practice it is confounded with label noise, with task difficulty, and with the format of the supervision.

We use exact verifiers to remove the first confound. When every label is correct, any difference between two training regimes on the same inputs is a difference in what the supervision exposes. We choose small decision tasks whose structure is known in advance (which surface rules approximate the label, which steps the natural procedure takes) so that what should be learnable can be stated before training. The point is not to teach a language model what a ten-line program already computes; it is that on such tasks we can tell which function the model learned, which is what a verifiable-reward pipeline for mathematics or code needs to know about its own supervision.

Answer-only supervision (arm A) teaches a shortcut. On 4-digit primality the answer-trained model reaches 82.3%, between a one-hot linear probe on the digits (79.3%) and the best cross-validated surface probe (89.7%), and its answers coincide with “the last digit is 1, 3, 7 or 9” on 99.6–100.0% of test items. Where no surface rule exists (divisibility by 7 or 13) it stays at chance at every training size we ran.

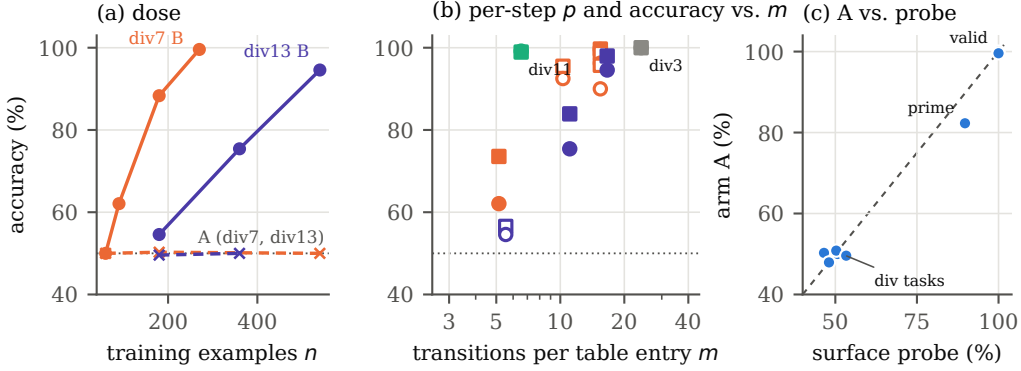


Figure 1: (a) Accuracy against training examples n for div7 and div13: trace arm B (solid) and answer arm A (dashed; chance at every n). (b) Arm B per-step accuracy p (squares; probability that a step is right given that the previous one was) and answer accuracy (circles) against $m = kn/(10d)$, the supervised transitions per (remainder, digit) table entry, known before training; orange div7 (4 and 6 digits), violet div13, green div11, gray div3. Hollow: cells observed before S7 was registered; filled: cells predicted in advance by S7 or S9. (c) Answer arm A against the surface probe fitted on the same training items (one-hot features where engineered features encode the answer); dashed line: equality.

Trace supervision (arm B: a deterministic procedure trace, then the answer) teaches the procedure once its steps are supervised often enough (Figure 1a). On divisibility by 7 the long-division trace takes accuracy from 50.0% at $n = 60$ training examples to 62.1%, 88.3% and 99.6% at 90, 180 and 270; on divisibility by 13, at the same trace length, from 54.6% at 180 to 75.4% and 94.6% at 360 and 540. The trace works only as the input’s own computation, and only if the model computes it before answering: a scrambled trace, a correct trace of a different number, and a trace placed after the answer all leave accuracy at chance, and prompting the base model does not come close.

What sets the dose? A trace for divisibility by d writes one remainder per digit, so n traces of k steps supervise $m = kn/(10d)$ transitions per entry of the (remainder, digit) table. After our first preregistered account of trace accuracy failed (S5, compounding a fixed per-step accuracy over length), we preregistered that per-step accuracy follows m (S7), a quantity known before training. Three of its predicted cells hold (Figure 1b); one fails because the step for $d = 11$ is easier ($10 \equiv -1 \pmod{11}$), and a preregistered dose test on $d = 13$ (S9) fails because its step is harder: at equal m , 13 trails 7. Supervision frequency drives trace learning within a task; step difficulty shifts the curve across tasks.

Contributions.

- A controlled comparison of supervision formats under exact labels, with scrambled-trace, ground-ing, answer-first, shuffled-label and prompting controls and paired tests on problem-disjoint test sets (§3, §4.3).
- Evidence that exact answer labels teach the surface probe’s function rather than the verifier’s (§4.1).
- Dose-response evidence that exact traces teach the procedure once its steps are supervised often enough, with a predictor fixed before training and a confound, step difficulty, that we name (§4.2).
- A preregistered record of ten tests, including three failures, all reported (Table 3).

One result reversed an earlier conclusion of ours. A verdict written on 2026-08-04 from a 60-example trace run stated that traces do not unlock divisibility by 7; the preregistered 180-example cell, which finished afterwards with the same code, gave 92.5%. The dose curve shows why both were right about their own n . The replication of that cell was a go/no-go condition for this submission, and it passed (90.0% and 82.5% in seeds 1 and 2).

Table 1: Tasks. k : steps in the trace (prime: for primes; a composite’s trace stops at its smallest factor). Tokens: mean whitespace tokens per trace. m : supervised transitions per (remainder, digit) table entry at $n = 180$.

task	trace procedure	k	tokens	m
prime	trial division by the primes $\leq \lfloor \sqrt{n} \rfloor$	11–25	71.5	–
div d , 4 digits	long division, $r \leftarrow (10r + x_t) \bmod d$	4	58.0	36.0 ($d=2$) ... 5.5 ($d=13$)
div7, 6 digits	same, $d = 7$	6	76.0	15.4
valid	Z3 countermodel or unsatisfiable core	certificate	37.0	–

2 RELATED WORK

Intermediate steps as supervision. Scratchpads (Nye et al., 2021) and chain-of-thought (CoT) prompting (Wei et al., 2022) showed that emitting intermediate steps improves multi-step computation, and CoT helps mainly on mathematical and symbolic tasks (Sprague et al., 2025). For small transformers trained from scratch, the format of arithmetic data changes sample efficiency sharply (Lee et al., 2024), and GCD predictions follow rules built from divisors of the number base (Char-ton, 2024), the kind of surface rule we probe for. Errors compound across the steps of compositional tasks (Dziri et al., 2023). Filler tokens can replace a chain of thought on some tasks (Pfau et al., 2024); our scrambled control keeps the tokens and removes their order, and on divisibility it removes the gain.

Theory. Parity-like targets give gradient-based learners almost no signal from end labels (Shalev-Shwartz et al., 2017). Sub-task decomposition makes otherwise unlearnable sequence tasks learnable (Wies et al., 2023); CoT raises the expressivity of bounded-depth transformers (Feng et al., 2023; Li et al., 2024; Merrill & Sabharwal, 2024), and next-token predictors trained on CoT data are universal learners (Malach, 2024). Transformers can represent automata through shallow shortcuts (Liu et al., 2023). The globality degree of Abbe et al. (2024) bounds what transformers learn efficiently as inputs grow, and an inductive scratchpad breaks that barrier; teacher-forced intermediate parities make parity learnable (Kim & Suzuki, 2025). Grokking studies how algorithmic tables such as modular arithmetic are learned from a fraction of their entries (Power et al., 2022). Section 6 uses these results as intuition only.

Traces and verifiers in post-training. Transformers trained from scratch on verifiable traces depend less on trace semantics than usually assumed (Valmeekam et al., 2026). Process supervision rewards steps rather than outcomes (Lightman et al., 2024); program-generated logic corpora improve reasoning (Morishita et al., 2024). STaR fine-tunes on self-generated rationales that reach verified answers (Zelikman et al., 2022); RL with verifiable rewards (RLVR) drives recent reasoning models (Lambert et al., 2024; DeepSeek-AI et al., 2025). Fine-tuning on model-generated positives can amplify spurious correlations (Setlur et al., 2024); SFT and RL generalize differently (Chu et al., 2025); RLVR mostly reweights what the base model already samples (Yue et al., 2025). Our answer arm is an instance of shortcut learning (Geirhos et al., 2020); exact labels do not prevent it.

3 SETUP

Tasks. Each task has an exact decision procedure, so labels are correct by construction; arms see the same prompts and labels and differ only in the completion (Table 1). **prime:** is a 4-digit n prime? The trace performs trial division by the primes up to $\lfloor \sqrt{n} \rfloor$, one “ $n \bmod p = m$.” step per prime, stopping at the first divisor. **div d :** is n divisible by d ? The trace performs long division, $r \leftarrow (10r + x_t) \bmod d$ for each digit x_t , for $d \in \{2, 3, 7, 11, 13\}$ with 4 digits and $d = 7$ with 6. **valid:** does a propositional conclusion follow from its premises (Z3 (de Moura & Bjørner, 2008))? Arguments instantiate 12 forms shared between training and test, so the task is surface-learnable by design (Appendix B). Test sets have 240 items (120 per class), are problem-disjoint from training and identical across arms; training sets are balanced samples of size n .

Arms. **base:** zero-shot. **A:** “Answer: Yes” or “Answer: No”. **B:** the trace, a newline, the answer (Appendix C). **C:** B’s traces permuted across examples and token-shuffled; the answer keeps the

Table 2: Accuracy (%) at $n = 180$, Qwen2.5-1.5B-Instruct, 240 test items per task; mean over seeds $\pm$ SD across seeds (seeds per cell: Appendix A; no SD: one seed so far). B–A: points, with a 95% bootstrap interval over items and seeds. max p : largest exact McNemar p (B vs. A) over seeds; McNemar treats the 240 items as the unit and so counts items, not training runs, for which the seed SD is the relevant uncertainty. Per-seed values: Appendix A.

task	base	probe	A	B	C	B–A [95% CI]	max p
prime	59.6	89.7	82.3 $\pm$ 0.3	55.8 $\pm$ 3.3	52.1 $\pm$ 2.9	-24.6 [-33.3, -15.6]	(5.7×10^{-7})
div7	62.5	46.5	50.3 $\pm$ 2.1	88.3 $\pm$ 5.2	50.0 $\pm$ 0.6	+38.1 [+31.0, +44.0]	7×10^{-13}
valid	30.0	100.0	99.6 $\pm$ 0.6	90.0 $\pm$ 14.1	52.5 $\pm$ 2.4	-9.6 [-23.3, +1.7]	$(0.50, 7.1 \times 10^{-15})$
div3	[TBD]	50.7	50.0	100.0	[TBD]	+50.0 [+43.8, +56.2]	
div11	[TBD]	50.3	50.8	99.2	[TBD]	+48.3 [+42.1, +55.0]	7.2×10^{-34}
div13	[TBD]	53.3	49.6	54.6	[TBD]	+5.0 [-5.8, +15.8]	0.41
div7 (6 digits)	[TBD]	48.1	47.9	93.1 $\pm$ 4.4	[TBD]	+48.3 [+41.7, +55.0]	1.1×10^{-32}

true label. **D** (S8): a fluent, correct trace of a *different* number with the same label. **B'**: answer first, then the trace. **A-full**: answers on the whole training pool. **S** (S10): the base model’s own sampled traces, kept when the exact gate accepts the final answer (STaR-style). **Shuffled labels**: from an earlier study with the same model and LoRA configuration. On the base model we also test a zero-shot CoT prompt and 4 trace demonstrations.

Training and evaluation. LoRA (Hu et al., 2022) with $r = 8$, $\alpha = 16$, dropout 0.05 on the attention projections of Qwen2.5-1.5B-Instruct (Qwen et al., 2024); AdamW, learning rate 2×10^{-4} , batch size 1, 3 epochs, bf16; loss on completion tokens only. Evaluation is greedy generation with one answer extractor for every arm. A run takes 1–17 minutes on one consumer GPU at $n = 180$.

Statistics, preregistration and provenance. Paired comparisons use the exact McNemar test (McNemar, 1947) within a seed, with Holm correction (Holm, 1979) across tasks at $\alpha = 0.05$; because McNemar counts test items, we also report seed SDs and bootstrap intervals over items and seeds. A first preregistration, frozen before a prior grid of 36 runs, fixed four endpoints (H1: B beats A; H2: B beats C; H3: B is more efficient per token; H4: on div7 B reaches about 60% with A at chance). The second was committed (b0370f5) at 20:29:47 on 2026-09-25, before any of its runs, and fixes S1–S6; its decision log adds, each before the cells it governs had finished, a clarification of S5’s primary cell (21:26:10), S7 (21:29:52), S8–S9 (21:38:05) and S10 (Appendix D). S3 and S6 were effectively settled before registration (S3 follows from the last-digit rule; A had already missed all 15 hard negatives of the prior test set). The surface probe belongs to neither preregistration. *Provenance*: the prior study’s verdict (2026-08-04) that traces do not unlock div7 was written from the 60-example run alone (50.0%) while the preregistered 180-example cell was still running; that cell finished at 92.5% with no code change.

4 RESULTS

4.1 EXACT ANSWER LABELS TEACH THE SURFACE

Answers track surface probes. We fit logistic regression, a one-hidden-layer MLP and k -nearest neighbours on the same 180 training items as arm A, on 40 one-hot digit-by-position features alone or with engineered divisibility features (last digit even or 0/5; digit sum mod 3 and 9; alternating digit sum mod 11), or for arguments on atom-renamed word and character n -grams, and choose the probe by 5-fold cross-validation; on div2, div3 and div11 the engineered features contain the answer, so there the probe is one-hot only. A tracks the fair probe (Figure 1c): on prime it lies between the one-hot linear probe (79.3%) and the best probe (89.7%), on every divisibility task both are at chance, and on valid both are near ceiling.

The primality shortcut. A’s answers (recovered from per-item correctness) coincide with “the last digit is 1, 3, 7 or 9” on 99.6% and 100.0% of test items, and that rule alone scores 82.5%: A has learned divisibility by 2 and 5 and nothing else (99.6% of primes, 65.0% of composites correct). The rule mislabels every composite ending in 1, 3, 7 or 9; the test set has 27 such composites with a

factor 3 or 7, easy for any sieve, and A gets 0 and 0 of them right (seeds 0 and 1). On a dedicated set of 120 odd composites without a factor ≤ 7 and as many primes (S6, passed, but settled in advance), A reaches 49.6% and agrees with the rule on 98.8% of items.

More answers do not buy the program. On prime, A reaches 59.6%, 82.3% and 75.0% at $n = 60, 180$ and 540 , and 80.8% on the full pool. On div7 it stays at chance at every n (50.0%, 50.3%, 50.0%) and on div13 likewise (49.6% and 50.0% at 180 and 360); on div7 it falls below the zero-shot base (62.5%; $p = 0.047, 0.0046, 6.5 \times 10^{-5}$ for seeds 0–2), whose long free-form answers answer-only training removes. With labels permuted across items, the earlier study’s answer arm sat at 48.5–52.5% on all seven of its tasks (Appendix A): correct labels drive A, and on primality what they buy is the rule.

4.2 EXACT TRACES TEACH STEPS SEEN OFTEN ENOUGH

Divisibility by 7. At $n = 180$, over three seeds, B reaches 88.3% (92.5, 90.0, 82.5) against A’s 50.3% ($p = 4.1 \times 10^{-27}, 2 \times 10^{-18}, 6.8 \times 10^{-13}$), and C stays at 50.0% ($p \leq 2 \times 10^{-20}$ against B). Seeds 1 and 2 are the preregistered replication (S1 passed); a re-run of seed 0 reproduced 92.5%. The gain holds on both classes (82.2% on multiples, 94.4% on non-multiples).

Dose. With the trace format fixed, accuracy rises with the number of training traces while A stays flat (Figure 1a). Div7 B goes from 50.0% at $n = 60$ to 62.1% at 90 ($p = 0.736$), 88.3% at 180 ($p = 0.955$) and 99.6% at 270 ($p = 0.997$). Div13 B, with the same 4 steps, goes from 54.6% at 180 ($p = 0.566$) to 75.4% at 360 ($p = 0.839$) and 94.6% at 540 ($p = 0.980$). At $n = 180$, div13’s failure is visible in its generations: 2.1% of traces are correct, the first error comes at step 2 in 188 of 240 items, and 179 of 235 wrong traces end at $r = 0$, so the model answers “yes” 76% of the time. In div7 generations, digits and carried remainders are copied without error (100.0%, 100.0%); every error is a wrong lookup of $(10r + x) \bmod 7$ (descriptively, lookups are right 0.947 of the time, $0.947^4 = 80.4\%$ against 82.9% error-free traces observed).

The predictor fixed before training (S7) and its confound. We preregistered that per-step accuracy p follows $m = kn/(10d)$, with five predicted cells, passing if four hold. Three hold: (a) div13 at $n = 540$ ($m = 16.6$): $p = 0.980$, accuracy 94.6%; (b) div7 at $n = 270$ ($m = 15.4$): $p = 0.997$; (c) div7 at $n = 90$ ($m = 5.1$): $p = 0.736$, accuracy 62.1%. Prediction (d) fails: div11 at $m = 6.5$ reached $p = 0.989$, above the predicted interval, because $10 \equiv -1 \pmod{11}$ makes its step a subtraction, $r' = (x - r) \bmod 11$ (div3, with $r' = (r + x) \bmod 3$, reaches 100.0%). Prediction (e) awaits div2. S7’s status, as computed by our analysis script, is pending (3/5 hold, 1 pending). The preregistered dose test S9 failed: div13 at $n = 360$ ($m = 11.1$) reached 75.4% against the required 80% (A: 50.0%), while div7 at the similar $m = 10.3$ reached 88.3%. At equal m , div13 trails div7: its step, $r' = (x - 3r) \bmod 13$, is harder, the same confound that failed (d) in the other direction. Within a task, supervision frequency drives trace learning; across tasks, step difficulty shifts the curve, and m alone does not predict it.

Length (S5): failed as preregistered. The prediction for 6-digit div7 was $B \approx p^6 = 75.8\%$, with $p = 0.955$ measured on 4-digit generations. The primary cell, B trained and tested on 6-digit traces, reached 96.2% and 90.0% in seeds 0 and 1, outside the 10-point tolerance: S5 failed. Its own per-step accuracy is 0.973, since at $k = 6$ the same n gives more transitions per entry ($m = 15.4$). The secondary cell, which holds the 4-digit model fixed and applies it to 6-digit inputs, gives 77.1%, close to p^6 : compounding describes length transfer for a fixed model, not what a model trained on longer traces learns. Forms that allow for a wrong trace ending in a right answer are post hoc (Appendix E).

Primality: the trace hurts. B reaches 55.8% against A’s 82.3% ($p = 5.7 \times 10^{-7}, 1.7 \times 10^{-7}$, [TBD] for seeds 0–2), and B vs. C is not significant ($p = 0.22, 0.33$). B classifies 19.7% of primes and 91.9% of composites correctly, but scrambled C shows the same split in seed 1 (20/120 primes, 110/120 composites), so the split reflects a bias toward “composite”, not trace length. At $n = 540$, B reaches 68.8% against A’s 75.0% ($p = 0.19$). A trial-division step is a table indexed by (n, p) with more than 10^5 entries, so $m \ll 1$ at these n .

Table 3: Preregistered tests and their status (details in Appendix D and Table 5).

	prediction	result	status
S1	div7 $B \geq 80$, $B - A \geq 20$ pp (seeds 1, 2)	B 90.0/82.5, A 50.8/47.9	passed
S2	traces help ≥ 2 of div3/11/13 (2 seeds)	$B - A + 50.0$, $+48.3$, $+5.0$ (seed 0)	pending
S3	div2 $A \geq 95$	100.0	passed (settled in advance)
S4	div7 $B > A$ at 3B	A [TBD], B [TBD]	pending
S5	6-digit div7 B within 10 pp of $p^6 = 75.8$	96.2/90.0 (secondary 77.1)	failed
S6	prime A on hard set < 60 , follows rule	49.6, rule agreement 98.8	passed (settled in advance)
S7	p follows m ; ≥ 4 of 5 cells	(a)–(c) hold, (d) fails, (e) pending	pending (3/5)
S8	arm D ≤ 60 (2 seeds)	50.0/50.4	passed
S9	div13, $n = 360$: $B \geq 80$, $A \leq 60$	B 75.4, A 50.0	failed
S10	arm S on div7 ≤ 65 ; valid ≥ 90	[TBD]	pending

Validity and cost. On valid both formats work (A 99.6%, B 90.0%, C 52.5%; $B - A$ $p = 0.50$, 7.1×10^{-15} , [TBD] for seeds 0–2). B’s completions contain 14–47 times as many tokens as A’s (Qwen tokenizer). Of the first preregistration’s endpoints, H1 held on div7 only, H2 on div7 and valid, H3 failed where answers work, and H4 held at $n = 180$ but not $n = 60$.

4.3 CONTROLS

The trace must be the input’s own computation. C (scrambled) sits at 50.0% on div7 and 52.5% on valid. D, a fluent and correct trace of a different number with the right label, gives 50.0% and 50.4% on div7 (S8 passed). What helps is the computation on this input, not trace-like text.

The trace must come before the answer. B’, trained with the trace after the answer, gives 48.3% and 50.8% on div7, like A, although it still writes a trace after answering with per-step accuracy 0.841. Training on traces placed after the answer does not move the procedure into the answer; the model has to compute the trace first.

Prompting does not replace training. On the base model, a zero-shot CoT prompt gives 67.1% on div7, 62.9% on prime and 66.7% on valid; 4 trace demonstrations give 47.9%, 39.2% and 68.8%. No prompt approaches trace-trained B on div7 (88.3%), so training, not inference-time reasoning alone, carries the gain. Arm S (S10, the base model’s own gate-filtered traces) gives [TBD]% on div7 (pending).

5 SCALE

Under the same protocol, Qwen2.5-3B-Instruct gives, as A/B: prime [TBD]/[TBD], div7 [TBD]/[TBD], valid [TBD]/[TBD] (S4 pending); div7 at 0.5B gives [TBD]/[TBD] and at 7B [TBD]/[TBD]. The surface probe does not depend on the model, so answer-only accuracy should stay near it at every size; how the dose curve moves with size is open.

6 AN AUTOMATON VIEW (INTUITION)

Remark 1 (intuition, not a theorem). Let $x_1 \cdots x_k$ be the digits of n and $\delta_d(r, a) = (10r + a) \bmod d$. Then $d \mid n$ iff $\delta_d^*(0, x_1 \cdots x_k) = 0$: divisibility by d is recognized by a d -state automaton reading digits. The arm-B trace writes its states. Under teacher forcing each supervised state is a function of two tokens already in the context, so n traces are kn samples of one table with $10d$ entries, $m = kn/(10d)$ per entry. Answer-only supervision gives n samples of the k -fold composition, and for d coprime to 10 the label is nearly independent of every proper subset of the digits: for $d = 7$ and $k = 4$, fixing any three digits leaves $\Pr(7 \mid n)$ between 0.10 and 0.22 (marginal 0.14).

The remark resembles the globality barrier of Abbe et al. (2024) and the teacher-forced CoT of Kim & Suzuki (2025), but those are statements about learning as the input length grows and do not apply literally to fixed 4-digit inputs. It counts how often each transition is supervised, not how hard it is: for $d = 3$ and $d = 11$ the transition is an addition or subtraction mod d , for $d = 7$ and $d = 13$ it involves a multiplication, and the results follow difficulty as well as m . Whether transformers learn such tables through the kind of shortcut Liu et al. (2023) describe, and how the rate depends on coverage of the table, as in grokking (Power et al., 2022), is what a model of the dose curve would have to explain.

7 DISCUSSION AND LIMITATIONS

What exact labels guarantee. An exact verifier guarantees that no training label is wrong, not that the labels identify the verifier’s function among those a model can fit. The rule A learned on primality calls every composite ending in 1, 3, 7 or 9 prime; by construction that includes all 996 of the 7,939 4-digit composites without a factor ≤ 7 . Shuffled labels show that the gain comes from label correctness but cannot tell the verifier’s function from a surface rule; inputs on which the rule is wrong can.

Supervision design for verifiable-reward pipelines. (1) Fit a cheap surface probe on the verified data before training; where it is accurate, answer supervision will teach its function, so evaluate where it is wrong. (2) A correct trace is not enough: budget how often each step type is supervised, expect harder steps to need more, and check per-step accuracy on held-out items after a small run. (3) The trace must be the input’s own computation and must precede the answer. (4) Do not read a null from one training size: our 60-example null on div7 was the bottom of a dose curve.

RLVR: a hypothesis, not a result. With a binary verifier the RLVR reward depends on the final answer only. We hypothesize, without testing, that RLVR then behaves like answer supervision with on-policy sampling: on a surface-learnable task a surface rule already earns most of the reward (the last-digit rule is right on 82.5% of our prime test items), and on div7 the policy would find the procedure only if its own samples sometimes execute it (Yue et al., 2025). We ran no RL; arm S (S10) is the nearest experiment.

Limitations. One model family, with 1.5B as the main size, and one LoRA configuration. Binary tasks with 240 test items; McNemar p -values count items, and with one to three seeds per cell the replication-level evidence is thinner than they suggest (div7 B has seed SD ± 5.2 points, valid B ± 14.1). The dose curves have one seed per n except $n = 180$. One trace format per task. S5 and S9 failed; S7 is pending with one failed cell, and m is confounded with step difficulty. S3 and S6 were settled before registration. The shuffled-label control comes from an earlier study; the probe was defined after the prior grid; the logic task shares its forms between training and test.

AI USE STATEMENT

We used an AI assistant (Claude, Anthropic) to write and refactor experiment code, to run the experiments and log their outputs under our direction, to compute the statistics, to draft and edit the manuscript, and to check every bibliography entry against arXiv, OpenReview, proceedings and DOI records. The research question, the arms, the preregistered decision rules and the interpretation were reviewed and approved by the authors before the runs they govern. We reviewed all AI-assisted code, text and numbers, verified reported numbers against the result files, and take responsibility for the content of this paper.

REPRODUCIBILITY STATEMENT

Code and data will be released: dataset generators with fixed seeds, the training and evaluation worker, queue runners, analysis scripts, both preregistration files with their commit history, and every run’s result record with per-item correctness, from which each paired test can be recomputed; an anonymized copy is in the supplementary material. Section 3 and Appendix C give prompts, trace formats and hyperparameters. Every number in the text is a macro regenerated from the result files (paper/exp/build_all.py).

ETHICS STATEMENT

The study uses synthetic arithmetic and logic data only and involves no human subjects or personal data.

REFERENCES

- Emmanuel Abbe, Samy Bengio, Aryo Lotfi, Colin Sandon, and Omid Saremi. How far can transformers reason? The globality barrier and inductive scratchpad. In *Advances in Neural Information Processing Systems*, 2024.
- François Charton. Learning the greatest common divisor: explaining transformer predictions. In *International Conference on Learning Representations*, 2024.
- Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V. Le, Sergey Levine, and Yi Ma. SFT memorizes, RL generalizes: A comparative study of foundation model post-training. In *International Conference on Machine Learning*, 2025.
- Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Lecture Notes in Computer Science, pp. 337–340. Springer, 2008. doi: 10.1007/978-3-540-78800-3_24.
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D. Hwang, Soumya Sanyal, Sean Welleck, Xiang Ren, Allyson Ettinger, Zaid Harchaoui, and Yejin Choi. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems*, 2023.
- Guhao Feng, Bohang Zhang, Yuntian Gu, Haotian Ye, Di He, and Liwei Wang. Towards revealing the mystery behind chain of thought: A theoretical perspective. In *Advances in Neural Information Processing Systems*, 2023.
- Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020. doi: 10.1038/s42256-020-00257-z.
- Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Juno Kim and Taiji Suzuki. Transformers provably solve parity efficiently with chain of thought. In *International Conference on Learning Representations*, 2025.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.
- Nayoung Lee, Kartik Sreenivasan, Jason D. Lee, Kangwook Lee, and Dimitris Papailiopoulos. Teaching arithmetic to small transformers. In *International Conference on Learning Representations*, 2024.
- Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma. Chain of thought empowers transformers to solve inherently serial problems. In *International Conference on Learning Representations*, 2024.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations*, 2024.

- Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Transformers learn shortcuts to automata. In *International Conference on Learning Representations*, 2023.
- Eran Malach. Auto-regressive next-token predictors are universal learners. In *International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 34417–34431, 2024.
- Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.
- William Merrill and Ashish Sabharwal. The expressive power of transformers with chain of thought. In *International Conference on Learning Representations*, 2024.
- Terufumi Morishita, Gaku Morio, Atsuki Yamaguchi, and Yasuhiro Sogawa. Enhancing reasoning capabilities of LLMs via principled synthetic logic corpus. In *Advances in Neural Information Processing Systems*, 2024.
- Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021.
- Jacob Pfau, William Merrill, and Samuel R. Bowman. Let’s think dot by dot: Hidden computation in transformer language models. *arXiv preprint arXiv:2404.15758*, 2024.
- Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.
- Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Amrith Setlur, Saurabh Garg, Xinyang Geng, Naman Garg, Virginia Smith, and Aviral Kumar. RL on incorrect synthetic data scales the efficiency of LLM math reasoning by eight-fold. In *Advances in Neural Information Processing Systems*, 2024.
- Shai Shalev-Shwartz, Ohad Shamir, and Shaked Shammah. Failures of gradient-based deep learning. In *International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 3067–3075, 2017.
- Zayne Sprague, Fangcong Yin, Juan Diego Rodriguez, Dongwei Jiang, Manya Wadhwa, Prasann Singhal, Xinyu Zhao, Xi Ye, Kyle Mahowald, and Greg Durrett. To CoT or not to CoT? Chain-of-thought helps mainly on math and symbolic reasoning. In *International Conference on Learning Representations*, 2025.
- Karthik Valmeekam, Vardhan Palod, Kaya Stechly, Atharva Gundawar, and Subbarao Kambhampati. Beyond semantics: The unreasonable effectiveness of reasonless intermediate tokens. *Transactions on Machine Learning Research*, 2026.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.
- Noam Wies, Yoav Levine, and Amnon Shashua. Sub-task decomposition enables learning in sequence to sequence tasks. In *International Conference on Learning Representations*, 2023.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems*, 2025.
- Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*, 2022.

A FULL RESULTS

Table 4 lists every run as logged. “B-ext” uses reasoning traces sampled from a locally served 9B model and kept only when their final answer matched the verifier (STaR-style rejection sampling); only 9 traces passed, so these runs are not comparable to B and are listed for completeness. Train tokens count whitespace tokens in the completions. Table 6 reports the earlier answer-only study with its shuffled-label control.

Table 4: Every run, as logged (240 test items per cell). Cell: the test set, prefixed by the training task when they differ; [cot] and [fewshot4] mark prompting controls on the base model. Train tokens: whitespace tokens in the completions.

model	cell	arm	n	seed	train tokens	accuracy (%)
1.5B	prime	base	0	0	0	59.6
1.5B	prime	A	60	0	120	59.6
1.5B	prime	A	180	0	360	82.1
1.5B	prime	A	180	1	360	82.5
1.5B	prime	A	540	0	1,080	75.0
1.5B	prime	B	60	0	4,355	57.1
1.5B	prime	B	180	0	13,370	57.9
1.5B	prime	B	180	1	13,260	57.5
1.5B	prime	B	180	2	13,080	52.1
1.5B	prime	B	540	0	40,095	68.8
1.5B	prime	C	180	0	12,561	50.0
1.5B	prime	C	180	1	13,120	54.2
1.5B	prime	A-full	1320	0	2,640	80.8
1.5B	prime	B-ext	9	0	896	62.1
1.5B	prime	B-ext	9	1	974	60.0
1.5B	prime→prime_hard	A	180	0	0	49.6
1.5B	prime[cot]	base	0	0	0	62.9
1.5B	prime[fewshot4]	base	0	0	0	39.2
1.5B	div7	base	0	0	0	62.5
1.5B	div7	A	60	0	120	50.0
1.5B	div7	A	180	0	360	52.1
1.5B	div7	A	180	1	360	50.8
1.5B	div7	A	180	2	360	47.9
1.5B	div7	A	540	0	1,080	50.0
1.5B	div7	B	60	0	3,600	50.0
1.5B	div7	B	90	0	5,400	62.1
1.5B	div7	B	180	0	10,800	92.5
1.5B	div7	B	180	1	10,800	90.0
1.5B	div7	B	180	2	10,800	82.5
1.5B	div7	B	270	0	16,200	99.6
1.5B	div7	C	180	0	10,784	49.6
1.5B	div7	C	180	1	10,824	50.4
1.5B	div7	B'	180	0	10,800	48.3
1.5B	div7	B'	180	1	10,800	50.8
1.5B	div7→div7_6d	B	180	0	0	77.1
1.5B	div7[cot]	base	0	0	0	67.1
1.5B	div7[fewshot4]	base	0	0	0	47.9
1.5B	valid	base	0	0	0	30.0
1.5B	valid	A	60	0	120	97.5
1.5B	valid	A	180	0	360	99.2
1.5B	valid	A	180	1	360	100.0
1.5B	valid	A	540	0	1,080	100.0
1.5B	valid	B	60	0	2,343	87.5
1.5B	valid	B	180	0	7,001	100.0
1.5B	valid	B	180	1	7,004	80.0
1.5B	valid	B	540	0	21,084	100.0
1.5B	valid	C	180	0	7,092	54.2
1.5B	valid	C	180	1	7,090	50.8
1.5B	valid	A-full	1600	0	3,200	100.0

model	cell	arm	n	seed	train tokens	accuracy (%)
1.5B	valid[cot]	base	0	0	0	66.7
1.5B	valid[fewshot4]	base	0	0	0	68.8
1.5B	div2	A	180	0	360	100.0
1.5B	div3	A	180	0	360	50.0
1.5B	div3	B	180	0	10,800	100.0
1.5B	div11	A	180	0	360	50.8
1.5B	div11	B	180	0	10,800	99.2
1.5B	div13	A	180	0	360	49.6
1.5B	div13	A	360	0	720	50.0
1.5B	div13	B	180	0	10,800	54.6
1.5B	div13	B	360	0	21,600	75.4
1.5B	div13	B	540	0	32,400	94.6
1.5B	div7_6d	A	180	0	360	47.9
1.5B	div7_6d	B	180	0	14,040	96.2
1.5B	div7_6d	B	180	1	14,040	90.0

Table 5: Preregistered cells in detail. Qwen2.5-1.5B-Instruct, 4 digits and $n = 180$ unless stated; accuracy in %; seed 0 unless stated. For S7 the B column is the per-step accuracy p ; for S8 and S10 it is the accuracy of arms D and S.

	cell	probe	A	B	criterion	met
S1	div7, s1 / s2	46.5	50.8 / 47.9	90.0 / 82.5	$B \geq 80$, $B-A \geq 20$ pp, $p < 0.01$ in each seed	passed
S2	div3	50.7	50.0	100.0	≥ 2 of the 3 with $B-A \geq 15$ pp in both seeds	pending
	div11	50.3	50.8	99.2		
	div13	53.3	49.6	54.6		
S3	div2	100.0	100.0	[TBD]	$A \geq 95$	passed
S4	div7, 3B model		[TBD]	[TBD]	$B > A$	pending
S5	div7 (6 digits), s0 / s1		47.9	96.2 / 90.0	$ B - p^6 \leq 10$ pp; $p^6 = 75.8$	failed
	secondary: 4-digit B			77.1	same	
S6	prime $\rightarrow$ hard set		49.6	[TBD]	$A < 60$; agrees more with the rule than with the label	passed
S7	(a) div13, $n = 540$			0.980	$p \geq 0.95$, acc. ≥ 85 (94.6)	holds
	(b) div7, $n = 270$			0.997	$p \geq 0.97$	holds
	(c) div7, $n = 90$			0.736	$p \leq 0.80$, acc. ≤ 75 (62.1)	holds
	(d) div11			0.989	$0.566 < p < 0.955$	failed
	(e) div3 / div2			1.000 / [TBD]	$p \geq 0.97$	pending
S8	div7, arm D, s0 / s1			50.0 / 50.4	$D \leq 60$ in both seeds	passed
S9	div13, $n = 360$		50.0	75.4	$B \geq 80$, $A \leq 60$	failed
S10	div7, arm S			[TBD]	$S \leq 65$ in seeds 0 and 1; valid	pending
					$S \geq 90$	

Table 6: Earlier answer-only study (same model and LoRA configuration; 240 training items, answer-only prompts, 3–4-digit numbers for arithmetic; mean of two seeds). Correct labels vs. labels permuted across training items.

task	base	correct labels	shuffled labels
divisible by 3	55.0	67.3	49.6
divisible by 7	53.3	51.0	52.5
divisible by 13	54.2	50.0	50.0
perfect square	51.2	83.3	51.5
prime	50.0	82.9	50.0
argument validity	66.7	99.4	50.0
joint consistency (SAT)	50.0	99.8	48.5

B THE LOGIC TASK

Arguments instantiate 6 valid forms (modus ponens, modus tollens, hypothetical syllogism, disjunctive syllogism, constructive dilemma, contraposition) and 6 invalid ones (affirming the consequent, denying the antecedent, affirming a disjunct, a broken chain, the converse, and “or” to “and”) with distinct random atom letters, rendered in English (“if A then B ”, “not A ”, “ A or B ”). The schema proposes an argument and Z3 decides it: the argument is labeled valid iff the premises together with the negated conclusion are unsatisfiable. For a valid argument the trace names the unsatisfiable core; for an invalid one it gives the satisfying assignment as a countermodel. Training and test share the 12 forms and never an instance, so a model can reach high accuracy by recognizing the form, which the atom-renamed n -gram probe does (100.0%). The task therefore checks that answers work when the surface determines the label; it is not a test of logical reasoning. Its results: base 30.0%, A 99.6% (99.2 and 100.0), B 90.0% (100.0 and 80.0), C 52.5% (54.2 and 50.8); at $n = 60$, A 97.5% and B 87.5%; at $n = 540$, both 100.0%.

C TRACE FORMATS

All arms share the prompt, which ends “End your reply with ‘Answer: Yes’ or ‘Answer: No’.” Examples of training completions, verbatim:

div7, B (prompt: “Is 1617 divisible by 7?”):

```
Process the digits of 1617 keeping a running remainder mod
7.  r = (10*0 + 1) mod 7 = 1.  r = (10*1 + 6) mod 7 = 2.  r
= (10*2 + 1) mod 7 = 0.  r = (10*0 + 7) mod 7 = 0.  Final
remainder 0, so 1617 is divisible by 7.
Answer:  Yes
```

div7, C (a scrambled trace from another example; the answer line keeps this example’s label):

```
running mod r = so = 4067 Process by 7.  7 a = 0.  0) 0.  7
mod (10*5 = r 7 the = = keeping + r is mod mod 4) (10*0 + of
4067 r + 7) 5.  = + digits remainder 7.  (10*0 = 7 0,
divisible Final 6) 4.  (10*4 mod remainder
Answer:  Yes
```

prime, B, a composite (“Is 4143 a prime number?”):

```
Check prime divisors up to isqrt(4143) = 64.  4143 mod 2 =
1.  4143 mod 3 = 0.  3 divides 4143, so 4143 is composite.
Answer:  No
```

prime, B, a prime (“Is 1493 a prime number?”):

```
Check prime divisors up to isqrt(1493) = 38.  1493 mod 2 =
1.  1493 mod 3 = 2.  1493 mod 5 = 3.  1493 mod 7 = 2.  1493
mod 11 = 8.  1493 mod 13 = 11.  1493 mod 17 = 14.  1493 mod
19 = 11.  1493 mod 23 = 21.  1493 mod 29 = 14.  1493 mod 31
= 5.  1493 mod 37 = 13.  No prime divisor up to 38, so 1493
is prime.
Answer:  Yes
```

valid, B, invalid (“Premises: (1) if H then D ; (2) D . Conclusion: H .”):

```
Assume every premise is true and the conclusion false.  That
is satisfiable:  set D=true, H=false.  Under this assignment
all premises hold while the conclusion is false, a
countermodel, so the argument is invalid.
Answer:  No
```

valid, B, valid (“Premises: (1) if S then R ; (2) S . Conclusion: R .”):

```
Assume every premise is true and the conclusion false, i.e.
 $\sim(R)$  holds.  Together these force a contradiction:   $\{S \rightarrow R,$ 
```

$S, \sim(R)\}$ cannot all hold at once. Since assuming the conclusion false is impossible, the conclusion follows.
 Answer: Yes

A (every task): Answer: Yes or Answer: No.

D PREREGISTRATION TEXT

First preregistration (prior grid). Frozen before any run of the prior grid and never edited; results were appended below it. It fixed: arms A, B, C, an answers-only arm at larger scale, arms using model-generated traces, and a conditional wrong-trace arm; Qwen2.5-1.5B-Instruct with the LoRA configuration of §3; tasks prime, valid and div7, all reported; 240 problem-disjoint test items per task (120 per class), identical across arms, graded by one parser on the final answer. Endpoints: H1, $B > A$ at $n = 180$ per task, paired McNemar, Holm across the three tasks; H2, $B > C$ at $n = 180$; H3, accuracy at matched training tokens; H4, on div7 B reaches about 60% with A at chance. The full text is in the supplementary material.

Second preregistration and its decision log. Reproduced from PREDICTIONS.md at the time of writing; Markdown markup converted, text unchanged except that the location in each time stamp is redacted. The header’s time was first written as an estimate (“~20:35”) and later replaced by the commit time; the decision log notes this.

Preregistered predictions and decision rules (frozen before any run listed here). Committed 2026-09-25 20:29:47 [location redacted] (commit b0370f5), before any of the runs below were started. The commit hash of this file is the preregistration. Earlier results (results/THINKING_VS_DATA.md, frozen 2026-08-04, and its later staged results block) are prior data, reported as such.

Provenance of the prior div7 result (stated in the paper). The 2026-08-04 verdict “H4 (a procedure trace unlocks div7): NO” was written from the $n = 60$ trace run alone ($B@60 = 50.0\%$). The preregistered main cell, $B@180$ seed 0, finished afterwards at 92.5% with the same worker (no code change). The paper reports the premature verdict and the later result.

Claim under test. Exact labels teach shortcuts; exact traces teach short programs. With labels from an exact verifier: answer-only supervision (arm A) recovers roughly what a simple probe on the answer surface recovers; trace supervision (arm B: procedure trace, then answer) succeeds when the per-step trace accuracy p , compounded over k steps, stays high (predicted accuracy about p^k), and fails when the procedure is long.

Decision rules. **S1 (go/no-go, before any other new run):** div7 $B@180$ seeds 1 and 2 each reach $\geq 80\%$ accuracy, with $B - A \geq 20$ percentage points and McNemar $p < 0.01$ against A of the same seed. If S1 fails, the paper is not submitted in this form. **S2:** at least 2 of {div3, div11, div13} (4-digit numbers) give $B - A \geq 15$ pp with the same sign in both seeds. **S3:** div2 $A \geq 95\%$ (the answer is surface-learnable from the last digit). **S4:** the div7 sign ($B > A$) holds for Qwen2.5-3B-Instruct. **S5 (falsifiable length test):** on 6-digit div7, B’s accuracy is within 10 pp of p^6 , where p is B’s per-step accuracy measured on 4-digit div7 generations. **S6 (shortcut):** A on prime agrees more with the rule “odd and no factor ≤ 7 ” than with the true label on hard negatives (odd composites with no factor ≤ 7), where A’s accuracy drops below 60%.

Definitions: a “win” is a difference of ≥ 10 pp with both seeds agreeing in sign; “neither” is both arms at or below majority class + 8 pp. McNemar tests per seed, Holm correction across tasks within each hypothesis family, $\alpha = 0.05$. Every cell is reported, including those that go against the claim.

Arms. base (zero-shot), A (answers), B (trace then answer), C (length-matched scrambled trace), B’ (answer then trace, div7 only), plus eval-only controls: zero-shot chain-of-thought prompt and 4-shot trace demonstrations on the base model. Model: Qwen2.5-1.5B-Instruct (laptop), Qwen2.5-3B-Instruct and others where the RTX 5090 allows. LoRA $r = 8$, $\alpha 16$, 3 epochs, $lr 2e-4$, bf16, greedy decoding, $n = 180$ training examples, 240 problem-disjoint test items.

Decision log.

2026-09-25 21:09:47 (commit fcaa05d) — **S1 PASS.** div7 $B@180$ seed 1 = 90.0% vs A 50.8% (+39.2 pp, McNemar $p = 2e-18$); seed 2 = 82.5% vs A 47.9% (+34.6 pp, $p = 6.8e-13$). Runs in results/thinking_vs_data/runs_L1.jsonl, produced by the unchanged worker

experiments/exp_thinking_ft_worker.py. The paper proceeds; the remaining tests run on experiments/exp_worker_v2.py.

2026-09-25 21:26:10 (commit 27ba807; the div7_6d B seed-0 generation file was written at 21:27:30) — **S5 clarification, recorded before either S5 cell finished** (div7_6d B seed 0 was mid-training; the 4-digit-adaptor transfer eval had not started). Primary S5 cell = arm B trained and tested on the 6-digit task (div7_6d), seeds 0 and 1, compared with p^6 where p = per-step accuracy of 4-digit div7 B generations (seed 0: $p = 0.955$, $p^6 = 0.758$). Secondary = the 4-digit B adaptor evaluated on 6-digit inputs. Also reported, labelled post hoc: the fraction of fully correct 6-digit traces vs p^6 , and answer accuracy vs $p^6 + (1 - p^6) * g$, where g is the answer-correct rate when the trace is wrong (4-digit: $g = 0.561$), since a wrong trace still yields the right yes/no answer about half the time.

2026-09-25 21:29:52 (commit 59e3989) — **S5 (primary) FAILED as preregistered**: 6-digit div7 B seed 0 = 96.2% vs $p^6 = 75.8\%$ (+20.4 pp > 10 pp tolerance). Its own per-step accuracy is 98.8% (vs 95.5% for the 4-digit model), so p is not a fixed property that transfers between training sets.

2026-09-25 21:29:52 (commit 59e3989; div11 B seed 0 started after 21:30:58) — **S7 (new hypothesis, preregistered before any of its cells ran)**: per-step trace accuracy p is set by the number of supervised transitions per entry of the (remainder, digit) table, $m = k * n / (10 * d)$ (k steps per trace, n training examples, $10d$ table entries). Observed so far: div7 $n=180$ $m=10.3 \rightarrow p=0.955$; div7_6d $n=180$ $m=15.4 \rightarrow p=0.988$; div13 $n=180$ $m=5.5 \rightarrow p=0.566$. Predictions: (a) div13 B $n=540$ seed 0 ($m=16.6$): $p \geq 0.95$ and answer accuracy $\geq 85\%$; (b) div7 B $n=270$ seed 0 (4-digit, $m=15.4$): $p \geq 0.97$; (c) div7 B $n=90$ seed 0 ($m=5.1$): $p \leq 0.80$ and answer accuracy $\leq 75\%$; (d) div11 B $n=180$ ($m=6.5$, not yet run): $0.566 < p < 0.955$; (e) div3 B $n=180$ ($m=24$) and div2 B $n=180$ ($m=36$): $p \geq 0.97$. S7 passes if at least 4 of (a)-(e) hold; every cell is reported either way.

Note (21:35): the clock times in the entries above were first written as estimates; they have been replaced by the commit timestamps, which are the authoritative record.

2026-09-25 21:38:05 (commit 1163562) **S8 and S9, preregistered before their cells ran** (div11 B seed 0 = 99.2% had finished; none of the cells below had started): S8 (grounding control): div7 arm D (a fluent, correct long-division trace of a DIFFERENT number with the same label) at $n = 180$, seeds 0 and 1: accuracy $\leq 60\%$ in both seeds, i.e. within 10 pp of A, far below B. S9 (dose, step difficulty fixed): div13 B at $n = 360$, seed 0 ($m = 11.1$ transitions per table entry, close to div7's 10.3 at $n = 180$): accuracy $\geq 80\%$; div13 A at $n = 360$ seed 0 stays $\leq 60\%$.

2026-09-25 21:46:44 (commit 8c0cfff) **S10, preregistered before any arm-S cell ran**. Arm S = self-generated traces filtered by the exact verifier on the final answer only (STaR / rejection-sampling fine-tuning, $K = 4$ samples per item at temperature 0.7 from the base model's chain-of-thought prompt; one kept completion per item; same 180 items and LoRA as A/B). Prediction: outcome-only verification does not supply correct steps, so on div7 arm S does not unlock the task: div7 S accuracy $\leq 65\%$ in seeds 0 and 1, and fewer than half of the kept div7 traces have all remainders correct. On valid, where answers are surface-learnable, $S \geq 90\%$. Prime S is reported without a prediction.

E THE COMPOUNDING APPROXIMATION (POST HOC)

This derivation is post hoc: it was not part of the preregistered S5 rule, and it does not change S5's verdict. Suppose each emitted state is correct with probability p , independently, and that after the first error the emitted final state is uniform on $\mathbb{Z}_d$. With probability p^k the trace is error-free and the answer is right. Otherwise the answer is right with probability $1/d$ on a multiple of d and $(d-1)/d$ otherwise, so on a balanced test set the accuracy is $p^k + \frac{1}{2}(1 - p^k) [\frac{1}{d} + \frac{d-1}{d}] = (1 + p^k)/2$. Both assumptions can fail: errors are correlated across steps (a model that miscomputes one table entry does so whenever the entry recurs), and a wrong state need not be uniform (on div13, wrong traces end at $r = 0$ in 179 of 235 cases). The positive-class accuracy, $p^k + (1 - p^k)/d$, is the cleaner check, because a model that answers "no" by default scores well on non-multiples: on 6-digit div7 it is 88.8%.